HEXORA

P2 — SPECIFICATION & PROTOTYPE

Video games, digital and physical · Week 7

Motroc Cosmin Alexandru
Catalog and admin lead · A1, A2, A9, A11

Nica Rares-Octavian
Payments and security lead · A5, A6, A8, A10

Iordachescu Iulian Daniel
Accounts and shopping lead · A3, A4, A7, A12

Repository
github.com/Iordachescu-Iulian/Project-E-commerce

Stack
Next.js 14 · React 18 · Tailwind · Node.js · MySQL 8.0 · Stripe test mode

Tag
P2 · E-Commerce 2026–2027

P2 AT A GLANCE

Working in the prototype

Home, catalog, category and product pages, search, register, login, logout and the address book — all reading MySQL, all server-rendered.

Proven, not mocked

Bcrypt passwords, signed httpOnly sessions, signature-verified Stripe webhooks, idempotent event storage and stock updates inside a transaction.

Frozen contracts

The ERD, 30 endpoints, five wireframes and the payment sequence are fixed here; weeks 8-12 implement cart, checkout, orders, admin and B4 + B5 against them.

13
tables, frozen Prisma schema

32
seeded editions · 16 games · 4 categories

9
test files, one command

What is frozen at P2

The data model, the API surface, the component architecture, the five wireframes and the payment flow. Changes after P2 require a written request on the course channel. The prototype proves the catalog, search, accounts and the Stripe webhook against real MySQL data.

What moves to P3

Cart, checkout, order history, stock UI, administration panel, full security enforcement, responsive audit, complete automated test suite and bonuses B4 + B5. Scope stays limited to A1–A12 plus the two approved bonuses.

AI tools are declared in the README: an AI coding agent assisted with scaffolding, code drafts and document drafts; every artifact was reviewed, run and tested by the team, and each member can explain the code they own.

REQUIREMENTS

A1–A12 and the approved bonuses

Every mandatory requirement is listed with the implementation that P3 will be graded against, the owner from P1 and the week in which it lands.

CODE	REQUIREMENT	HOW IT IS IMPLEMENTED	OWNER	STATUS
A1	**Product catalog**	Server-rendered paginated catalog capped at 24 products, a page per category, and a product page with name, description, gallery, price, platform, format, stock and the sibling edition of the same game.	Cosmin	DONE
A2	**Basic search**	Case-insensitive search on product names; filters and query survive in the URL; a clear “No games found” state with a reset link.	Cosmin	DONE
A3	**Shopping cart**	One cart per authenticated user; add, update and remove; totals recomputed on the server; quantities above stock rejected; cart persists after logout.	Iulian	WEEK 8
A4	**User accounts**	Register, login and logout with bcrypt (cost 12); a signed HS256 JWT in an httpOnly, SameSite=Lax cookie; address CRUD with exactly one default address.	Iulian	DONE
A5	**Checkout**	Address → delivery method (digital 0 lei, courier 19,99 lei) → server-validated order summary → Stripe card payment; confirmation page displays the order number.	Rares	WEEKS 8–9
A6	**Sandbox payment**	An order becomes Paid only after a signature-verified Stripe webhook; a browser redirect never changes status; a failed payment stays Unpaid and is visible in PaymentEvent.	Rares	WEBHOOK
A7	**Stock management**	Stock is decremented inside the paid transaction with a conditional `stock >= quantity` update; a zero-row update rolls back; cancellation restores stock exactly once.	Iulian	SERVICE
A8	**Order history**	Customer order list and detail with date, items, total, payment state and status history.	Rares	WEEK 9
A9	**Administration panel**	Separate ADMIN role; product and category CRUD with image upload; order list filterable by status; audited transitions New → Paid → Shipped → Delivered, or Cancelled.	Cosmin	WEEKS 10–11
A10	**Basic security**	Zod schemas on every mutating route, same-origin check, security headers, server-side `requireUser`/`requireRole` guards and Prisma parameterisation.	Rares	GROUNDWORK
A11	**Responsive design**	All customer pages usable at 360 px: 2-column product grid on mobile, 4-column desktop, no horizontal scroll; screenshots captured at both widths.	Cosmin	DONE
A12	**Automated tests**	Vitest with one command; payment confirmation, stock rollback and restoration, password/session, catalog, money and pagination covered. Cart and total tests land with A3.	Iulian	GROWING

Approved bonuses — only these two

CODE	BONUS	IMPLEMENTATION AND ACCEPTANCE	POINTS	STATUS
B4	**Sales dashboard**	Admin dashboard for a selected interval: daily sales chart, top five products by revenue and average order value, computed with grouped SQL aggregates over paid orders.	3	WEEK 11
B5	**Discount codes**	Percentage or fixed amount, optional expiry and maximum uses, validated on the server at checkout; the used counter increments once per paid order; admin CRUD.	3	WEEK 11

Bonus points are only awarded after A1–A12 work; both bonuses therefore land in week 11, after checkout and the admin panel.

19
live checks pass against the running app — `scripts/verify-app.mjs`

9
Vitest files: payments, stock, auth, sessions, catalog, money, pagination

14
screenshots at 1440 px and 360 px in `docs/screenshots`

What was verified by hand

- Register → login → create an address → logout → login again shows the saved address.
- A signed webhook marks the order paid and decrements stock once; replaying it returns `duplicate: true`.
- An invalid or missing signature returns 400 and leaves the order untouched.
- An unknown order id rolls the transaction back; no order becomes paid.

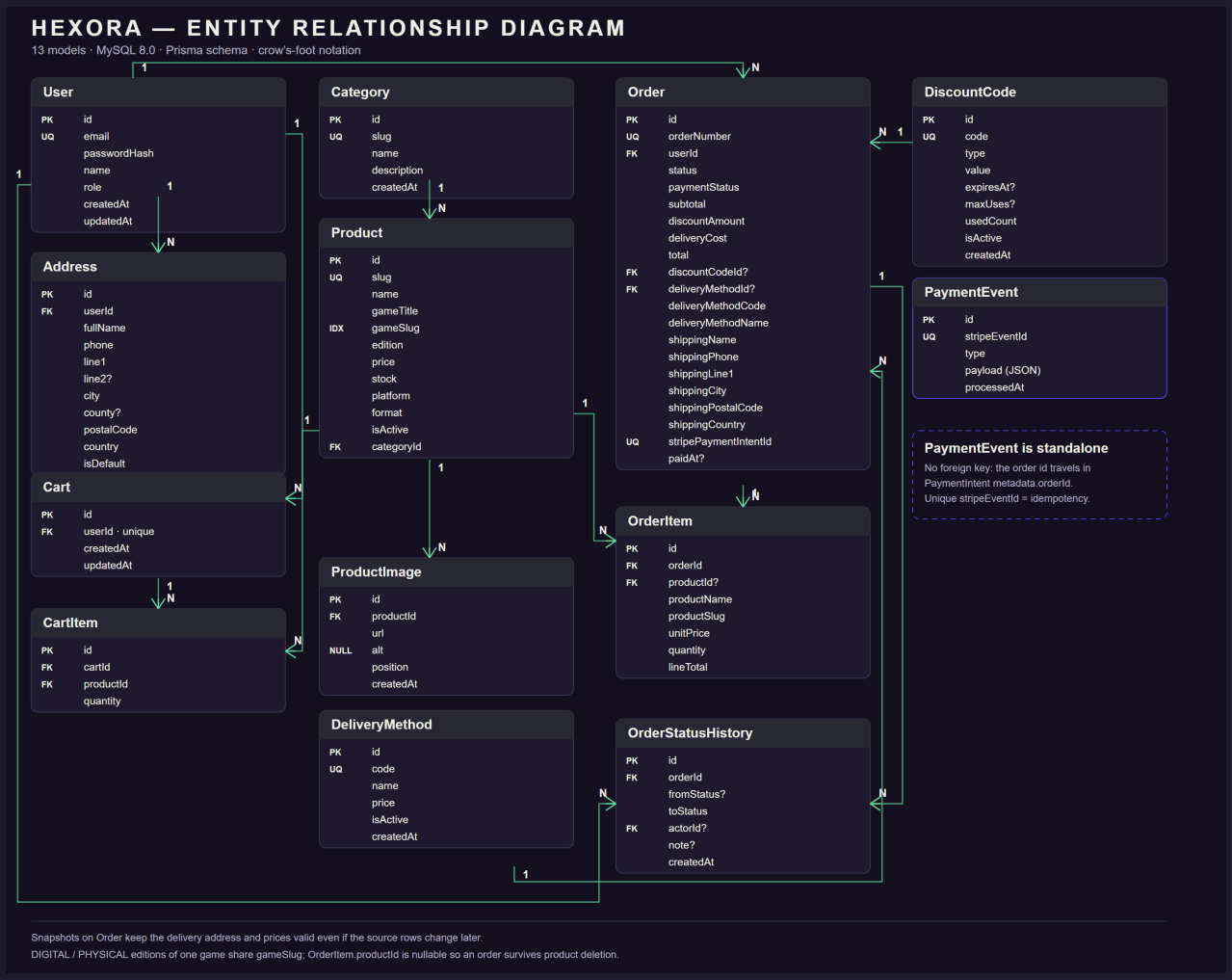
Out of scope at P2

- Cart, checkout screen and order history (weeks 8–9); the endpoints exist only as frozen contracts.
- Admin screens; the ADMIN role and the server guards exist, the pages arrive in weeks 10–11.
- Real card charges: Stripe test mode only, no live keys anywhere in the repository.

DATA MODEL

Entity–relationship diagram

13 models in MySQL 8.0. A Product is one purchasable edition; the digital and physical editions of a game share `gameSlug` and are shown together.



Key relationships

- **User 1—N Address / Order**; one optional Cart per user, one CartItem per product.
- **Category 1—N Product 1—N ProductImage / CartItem / OrderItem.**
- **Order 1—N OrderItem** and **Order 1—N OrderStatusHistory**; delivery and discount are optional FKs.
- **OrderStatusHistory.actorId** references the administrator who changed the status.

Rules the model enforces

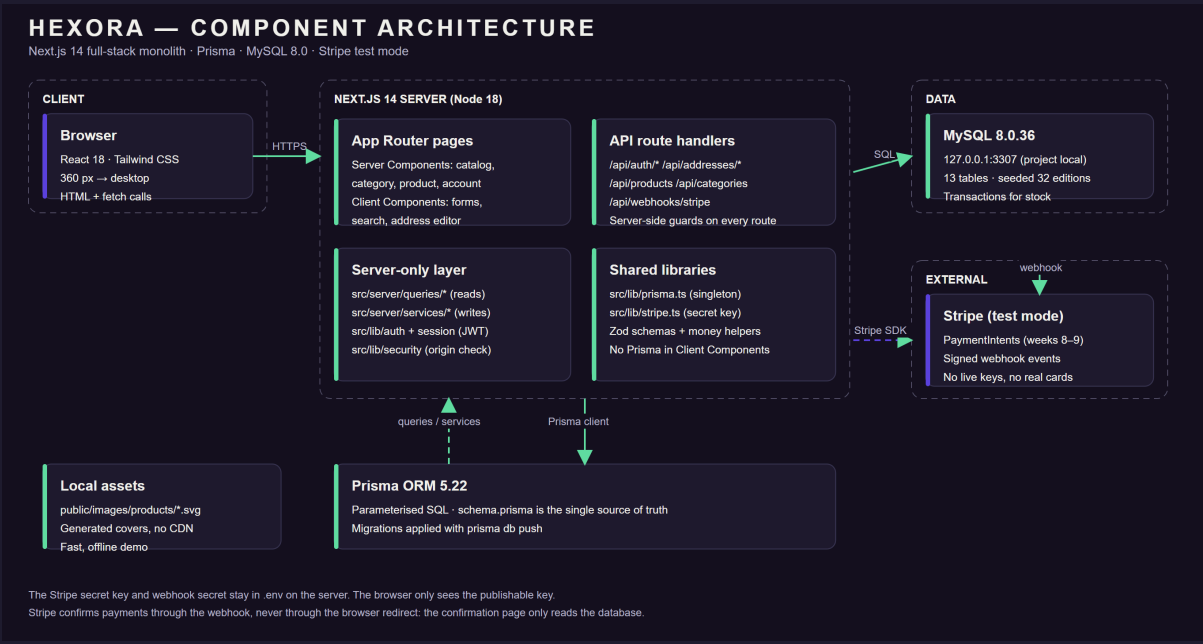
- **Order snapshots:** shipping fields, product name, slug and unit price are copied at checkout so later edits cannot rewrite history.
- **Nullable OrderItem.productId:** deleting a product keeps past orders readable.
- **PaymentEvent.stripeEventId is unique:** the database itself guarantees webhook idempotency.
- **Money is Decimal(10,2)**; dates are server timestamps; all foreign keys are indexed.

Constraints the database enforces

CONSTRAINT	WHERE	WHY IT IS THERE
Unique	User.email · Category.slug · Product.slug · Order.orderNumber · DiscountCode.code	One account per email, one stable public URL per product and per category.
Unique	PaymentEvent.stripeEventId · Order.stripePaymentIntentId · Cart.userId · CartItem(cartId, productId)	Idempotent webhooks, one intent per order, one open cart and one line per product.
Cascade / Restrict	Children cascade with their parent; Product.categoryId, CartItem.productId and Order.userId restrict deletion	Rows that cannot exist alone disappear; a category in use or a customer with orders cannot vanish.
SetNull	OrderItem.productId · Order.discountCodeId · Order.deliveryMethodId · OrderStatusHistory.actorId	Deleting a product, code, method or administrator never rewrites order history.
Index	Product.name, gameSlug, categoryId, isActive · Order.status, paymentStatus, userId+createdAt, paidAt	Catalog filters and search, admin status filters and the B4 dashboard interval.

ARCHITECTURE

Components, stack and startup



Responsibilities

- **Server Components** render the catalog, category, product and account pages; data comes from `src/server/queries/*`.
- **Client Components** own the search box, auth forms, address editor and future cart interactions.
- **API route handlers** validate input with Zod, enforce auth/roles and call `src/server/services/*`.
- **Prisma singleton** is the only database entry point; no Prisma import reaches a Client Component.

Stack justification

- **Next.js + React + Tailwind:** one language and one process, responsive by default, accepted without prior approval.
- **MySQL 8.0.36:** relational transactions for orders and stock; a project-local instance on port 3307 keeps the demo self-contained.
- **Stripe test mode:** no live keys or real cards; signed webhooks prove the payment state server-side.
- **Local SVG covers:** no external image dependency during the live demo.

Startup — at most five commands

```
npm install
npm run db:start
npm run db:push
npm run db:seed
npm run dev
```

Or `npm run setup` followed by `npm run dev`. Secrets live in `.env`; only `.env.example` is committed. App: `http://localhost:3000`, MySQL: `127.0.0.1:3307`.

Demo accounts, seeded by `prisma/seed-users.ts`

Customer · demo@hexora.test · Demo1234!

Administrator · admin@hexora.test · Admin1234!

Both rows store bcrypt hashes; the passwords above exist only in the seed script and this document.

Commands that reproduce the demo

npm test · 9 files

node scripts/verify-app.mjs · 19 checks

node scripts/test-webhook.mjs · signed events

The verification script needs the dev server on `localhost:3000`; the webhook script needs both Stripe variables in `.env`.

Environment — six variables, none of them committed

VARIABLES	NEEDED FOR	NOTES
`DATABASE_URL` · `AUTH_SECRET` · `STRIPE_SECRET_KEY` · `STRIPE_WEBHOOK_SECRET`	Server only	MySQL on `127.0.0.1:3307`, the HS256 session key, and the test keys `sk_test_…` / `whsec_…`. Missing Stripe values make the webhook answer 503 rather than fail silently.
`NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY` · `NEXT_PUBLIC_APP_URL`	Browser	The only public values: the test publishable key and the origin compared against `Origin` on mutating requests.

API SURFACE

Endpoints, methods and access levels

Access is enforced on the server. The first table is implemented at P2; the second is frozen now and delivered in weeks 8–11.

Implemented at P2

METHOD	ROUTE	ACCESS	PURPOSE
GET	/api/products	public	Paginated catalog, max 24/page; q, category, page, platform, format, sort
GET	/api/products/[slug]	public	Product detail with images and sibling editions
GET	/api/categories	public	Categories with active edition counts
POST	/api/auth/register	public	Validate, hash password, create user and session
POST	/api/auth/login	public	Verify password and set the signed cookie
POST	/api/auth/logout	authenticated	Clear the session cookie
GET	/api/auth/me	authenticated	Current user or 401
GET	/api/addresses	customer	List own addresses, default first
POST	/api/addresses	customer	Create address; the first one becomes default
PATCH	/api/addresses/[id]	owner	Update address or promote it to default
DELETE	/api/addresses/[id]	owner	Delete address; promote the next one
POST	/api/webhooks/stripe	Stripe signature	Verify, record once, mark order paid, decrement stock

Frozen for P3

METHOD	ROUTE	ACCESS
GET	/api/cart	customer
POST	/api/cart/items	customer
PATCH	/api/cart/items/[id]	owner
DELETE	/api/cart/items/[id]	owner
POST	/api/checkout	customer
GET	/api/orders	customer
GET	/api/orders/[id]	owner
GET	/api/delivery-methods	public
POST	/api/discounts/validate	customer

METHOD	ADMIN ROUTE	ACCESS
GET/POST	/api/admin/products	ADMIN
PATCH/DELETE	/api/admin/products/[id]	ADMIN
POST	/api/admin/products/[id]/images	ADMIN
GET/POST	/api/admin/categories	ADMIN
PATCH/DELETE	/api/admin/categories/[id]	ADMIN
GET	/api/admin/orders	ADMIN
PATCH	/api/admin/orders/[id]/status	ADMIN
GET	/api/admin/dashboard	ADMIN · B4
GET/POST/PATCH	/api/admin/discounts	ADMIN · B5

Access model: `requireUser` for authenticated routes, `requireRole('ADMIN')` for the admin surface, ownership checks on address and order ids, and `assertSameOrigin` on every mutating browser request.

Response — `GET /api/products?q=crown&page=1`

```
{
  "items": [{
    "id": "clx…", "slug": "ashen-crown-digital",
    "name": "Ashen Crown — Digital",
    "gameTitle": "Ashen Crown",
    "gameSlug": "ashen-crown",
    "edition": "DIGITAL", "platform": "PC",
    "format": "DIGITAL", "price": "129.99",
    "stock": 42, "imageUrl": "/images/…svg",
    "category": { "slug": "action-adventure",
                  "name": "Action & Adventure" }
  }],
  "page": 1, "pageSize": 24,
  "total": 2, "totalPages": 1
}
```

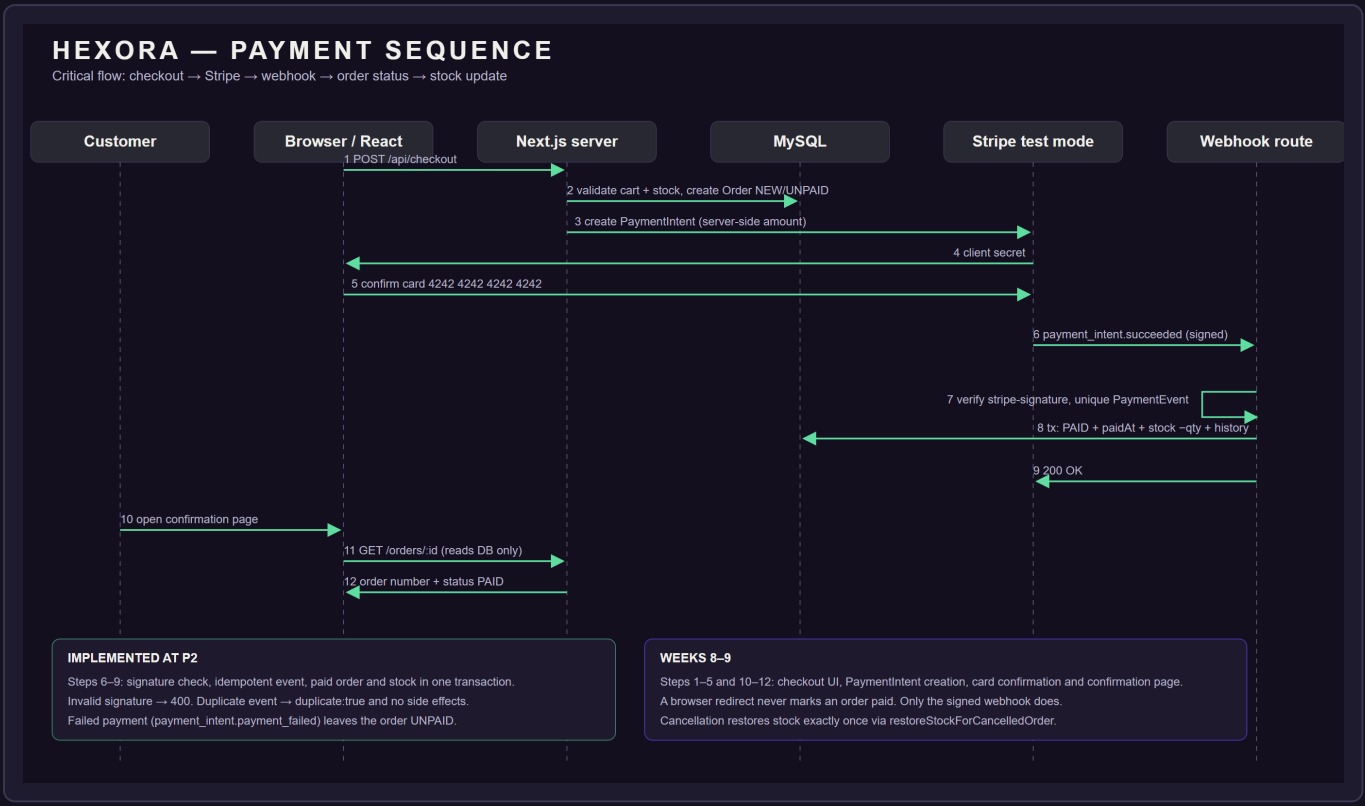
Error contract — every handler

```
{ "error": "Invalid Stripe signature." }
```

- **400** validation or signature failure · **401** no session · **403** wrong role or foreign address.
- **404** unknown slug, product or order · **409** duplicate email or stock conflict.
- **500** unexpected failure · **503** Stripe not configured.
- Money is serialised as a decimal string, never a float, so `129.99` cannot become `129.989999` in JSON.
- `pageSize` is clamped to 24 server-side; a larger value is accepted but capped, never trusted.

CRITICAL FLOW

Payment sequence and stock update



Steps 1–6 — checkout

- The server reloads the cart, validates stock and totals, creates an Order with status `NEW` and payment `UNPAID`, then creates a PaymentIntent with the server-computed amount.
- The browser confirms the test card with Stripe.js. A redirect back to the site never changes the order: only the webhook does.

Steps 7–9 — webhook (implemented)

- Raw body + `stripe-signature` verified with `constructEvent`; invalid → 400.
- `PaymentEvent.stripeEventId` is unique, so a duplicate delivery returns `duplicate: true` and does nothing else.
- One transaction marks the order Paid, sets `paidAt`, decrements stock conditionally and appends status history.

Failure and cancellation

- `payment_intent.payment_failed` stores the event only; the order remains `UNPAID`.
- If any stock decrement affects zero rows, the transaction rolls back and the order stays unpaid; Stripe retries.
- Cancelling an order calls `restoreStockForCancelledOrder`, which increments stock exactly once and records the transition.

Verification at P2

- A bad signature returns 400; a missing signature returns 400.
- `scripts/test-webhook.mjs` sends a signed event twice: first `duplicate:false`, then `duplicate:true`.
- An unknown order id is rejected and no order is marked paid.
- Service tests cover paid marking, stock rollback, double-payment and single restoration.

Webhook HTTP contract — `POST /api/webhooks/stripe`

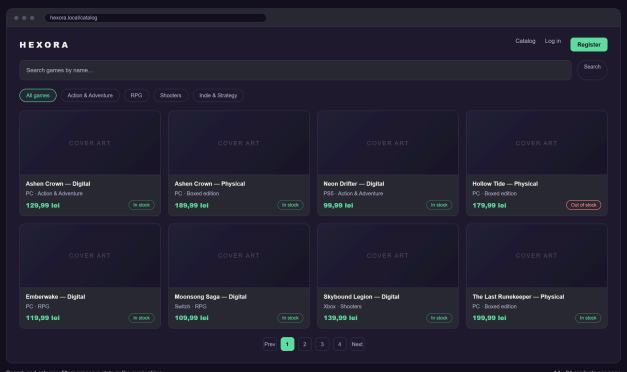
STATUS	WHEN	BODY
200	Event is new	`{ "received": true, "duplicate": false }` · order paid, stock decremented, history appended
200	Event already stored (Stripe retry)	`{ "received": true, "duplicate": true }` · nothing is written twice
400	`stripe-signature` header missing	`{ "error": "Missing Stripe signature." }`
400	`constructEvent` rejects the payload	`{ "error": "Invalid Stripe signature." }`
500	Transaction fails, for example an unknown order id	`{ "error": "Stripe webhook could not be processed." }` · the event row is rolled back so Stripe can retry
503	`STRIPE_SECRET_KEY` or `STRIPE_WEBHOOK_SECRET` absent	`{ "error": "Stripe webhook is not configured…" }`

USER INTERFACE

Five wireframes

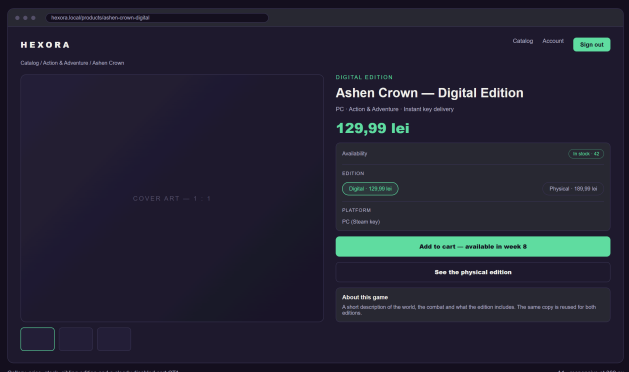
Catalog, product page, cart, checkout and administration panel. Every customer screen keeps the same hierarchy at 360 px; the catalog grid collapses from four columns to two.

CATALOG — WIREFRAME · A1 · 1440 PX



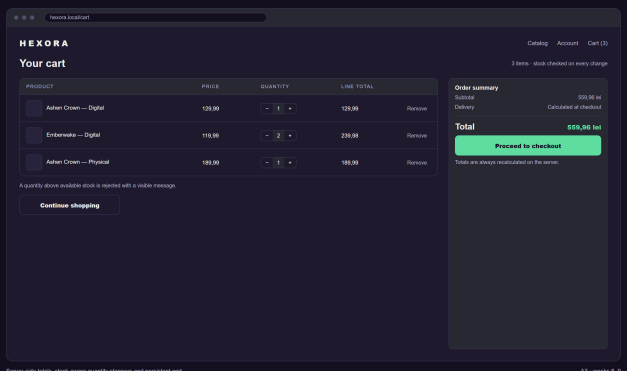
CATALOG · A1 · SEARCH, FILTERS, 24/PAGE

PRODUCT PAGE — WIREFRAME · A1 · 1440 PX



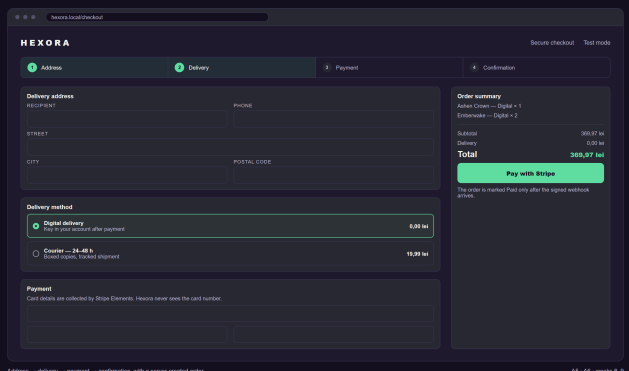
PRODUCT PAGE · A1 · GALLERY, STOCK, SIBLING EDITION

CART — WIREFRAME · A3 · 1440 PX



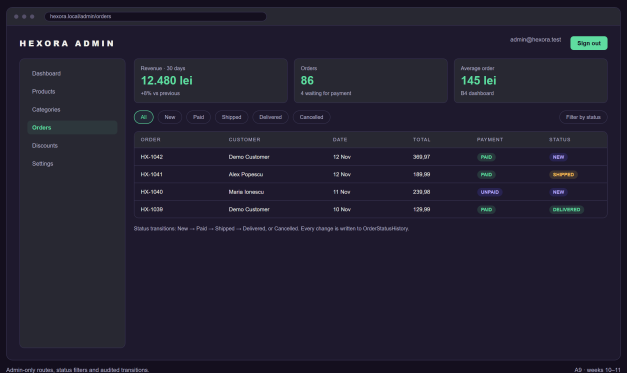
CART · A3 · SERVER TOTALS, STOCK CAPS

CHECKOUT — WIREFRAME · A5 / A6 · 1440 PX



CHECKOUT · A5 / A6 · ADDRESS, DELIVERY, STRIPE

ADMIN PANEL — WIREFRAME · A9 · 1440 PX



ADMINISTRATION · A9 · ORDERS, FILTERS, TRANSITIONS

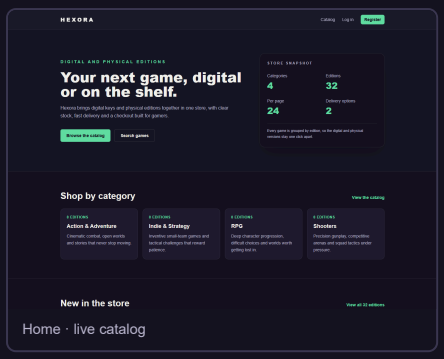
Shared layout

Dark theme from P1 (#14121F), cream text (#F4F2EC), mint accent (#5FE0A4), violet secondary (#5B45E0), Arial. One header, one footer and a 6xl container on every customer page.

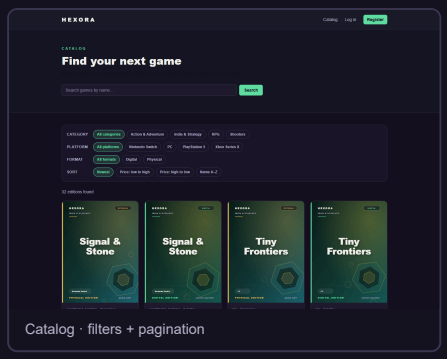
The disabled cart button on the product page is intentional until A3 lands; the label says so instead of pretending the action exists.

PROTOTYPE, SCHEDULE AND ACCEPTANCE

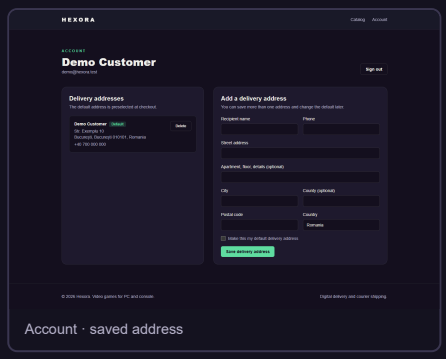
Live prototype, weeks 8–12, definition of done



Home · live catalog



Catalog · filters + pagination



Account · saved address

Schedule

WEEK	WORK	OWNER	STATE
3	P1, repository, README	All	DONE
4	ERD, endpoints, wireframes	All	DONE
5	Seed · A1 catalog · A4 accounts	Cosmin, Iulian	DONE
6	A2 search · Stripe setup · login	Cosmin, Rares	DONE
7	P2 specification and live prototype	All	DONE
8	A3 cart · A5 checkout	Iulian, Rares	NEXT
9	A6 confirmation · A7 stock · A8 history	Rares, Iulian	PLAN
10	A9 admin · A10 security	Cosmin, Rares	PLAN
11	A11 responsive · A12 tests · B4 + B5	Cosmin, Iulian	PLAN
12	P3 fixes, backup video, final README	All	PLAN

Top risks

- **Payments and stock:** webhook-first, conditional updates, rollback and transaction tests.
- **Integration:** frozen contracts, small merges, one integration branch before the demo.
- **Scope:** required features first, feature freeze in week 11, backup video.

Branches and reviews

MEMBER	BRANCH	CONTENTS	REVIEWED BY
Iulian	`p2/iulian-accounts`	Auth, session, addresses	Cosmin, Rares
Cosmin	`p2/cosmin-catalog-search`	Seed, catalog, search	Rares, Iulian
Rares	`p2/rares-payments-security`	Webhook, stock, security	Cosmin, Iulian
All	`p2/integration`	Merge order Iulian → Cosmin → Rares, then `main` + tag `P2`	Three-person review

Demo accounts

Customer demo@hexora.test / Demo1234!

Administrator admin@hexora.test / Admin1234!

Seeded by `prisma/seed-users.ts`; passwords are bcrypt hashes.

Definition of done for P2

- Setup + dev starts the store locally; seed creates 4 categories, 16 games, 32 editions with images, prices and stock.
- Catalog is paginated at 24, category and product pages work, the sibling edition is visible.
- Search returns matches and a clear empty state.
- Register, login, logout and addresses work; `/account` is blocked without a session.
- Webhook rejects invalid signatures and stores a valid event once.
- Pages are usable at 360 px.
- Three member branches, PR descriptions and a `P2` tag exist; README lists at most five startup commands.

AI use

An AI coding agent (OpenCode) assisted with project scaffolding, code and document drafts. The team reviewed every change, ran the tests and the app, and can explain the code. The README repeats this declaration.

Deliverables in the repository

- **Specification** — this document, 8 A4 pages.
- **Deck** — 15 slides, `P2 - Hexora.pdf` and `.pptx`.
- **Diagrams** — ERD, architecture and payment sequence, SVG and PNG.
- **Wireframes** — five screens at 2× in `docs/deliverables`.
- **Screenshots** — 14 captures at 1440 px and 360 px.
- **Docs** — README, `docs/payments.md`, `docs/P2-schedule.md`, `docs/PR-DESCRIPTIONS.md`.